Database Setup Guide: Intelligent Music Recommendation System

This guide is designed to help team members get the Intelligent Music Recommendation System up and running quickly. It breaks down the architecture and provides a step-by-step setup process.

1. Project Architecture

The repository is organized into three specialized modules to ensure separation of concerns:

```
Intelligent-Music-Recommendation-and-Song-Similarity-System
│
├── backend           # Infrastructure & Environment
├── db                # Database Schema & Migrations
└── similarity_model  # Machine Learning Logic don't need to download
```

2. Folder Breakdown

Folder	Responsibility	Key Files
backend	The Engine Room. Handles the database server and Python dependencies.	docker-compose.yml, requirements.txt
db	The Blueprint. Defines how data is stored and manages updates.	models.py, alembic/, test_db.py
similarity_model	The Brain. Contains the AI logic for song similarity.	Feature extraction & training scripts

3. Getting Started (Setup Guide)

Step 1: Clone & Prepare

```
git clone <repo-url>
cd Intelligent-Music-Recommendation-and-Song-Similarity-System
```

Step 2: Launch the Database (Docker)

```
cd backend
docker compose up -d
```

Step 3: Setup Python Virtual Environment

```
cd ..
python -m venv venv

# Windows:
venv\Scripts\activate
# Mac/Linux:
source venv/bin/activate
```

Step 4: Install Dependencies

```
pip install -r backend/requirements.txt
```

Step 5: Apply Database Schema (Alembic)

```
cd db
python -m alembic upgrade head
```

Step 6: Verify the Connection

```
python test_db.py
# Expected: DB connection OK, SELECT 1 -> 1
```

Step7: Connect DataGrip:

```
Host: localhost
Port: 3306
User: app_user
Password: app_password
Database: project_db
```

4. Troubleshooting & Tips

- Docker: Ensure Docker Desktop is running if the command fails.
- Port Conflicts: If port 3306 is busy, stop any local MySQL services.
- Alembic: Run 'alembic upgrade head' after every git pull.